Document number: P0641R1
Date: 2017-07-24
Project: Programming Language C++
Audience: Evolution Working Group
Reply-to: Daniel Krügler, Botond Ballo

Resolving Core Issue #1331 (`const` mismatch with defaulted copy constructor)

Revision History

Changes since P0641R0:

- Updated proposed wording to address comments from CWG review.
- Fixed outdated section numbers and typos.

Motivation

If you have a class with a copy constructor whose parameter type is reference-to-nonconst:

```
struct MyType {
    MyType(MyType&);  // no 'const'
};
```

and you try to aggregate it in a class that has a defaulted copy constructor with the usual signature:

```
template <typename T>
struct Wrapper {
    Wrapper(const Wrapper&) = default;
    T t;
};

Wrapper<MyType> var;  // fails to instantiate
```

the resulting type is ill-formed, even if it is never used in a context where the copy constructor is required.

The authors believe this is unnecessarily restrictive; one should be able to use `Wrapper<MyType>` as long as one does not try to copy it.

`std::tuple` is an example of a wrapper type which suffers from this problem in popular implementations.

Analysis

The relevant standard wording can be found in 11.4.2 [dcl.fct.def.default] paragraph 1 (emphasis ours):

> 1. [...] A function that is explicitly defaulted shall:
>   - be a special member function,
>   - have the **same declared function type** (except for possibly differing ref-qualifiers and except that in the case of a copy constructor or copy assignment operator, the parameter type may be "reference to non-const T", where T is the name of the member function's class) **as if it had been implicitly declared**, and
>   - not have default arguments.

In the case of `Wrapper<MyType>`, the implicitly declared copy constructor would have the parameter type `Wrapper<MyType>&` per 15.8.1 [class.copy.ctor] paragraph 7, so an explicitly defaulted copy constructor with the parameter type `const Wrapper<MyType>&` is ill-formed.

Proposed Resolution

This proposal suggests that if the declared type of an explicitly defaulted function is not the same as if it had been implicitly declared, then it be defined as deleted, rather than being ill-formed.

Proposed Wording

Change 11.4.2 [dcl.fct.def.default] paragraph 1 as indicated:

> 1. [...] A function that is explicitly defaulted shall:
>   - be a special member function, and
>   - ~~have the same declared function type (except for possibly differing ref-qualifiers and except that in the case of a copy constructor or copy assignment operator, the parameter type may be "reference to non-const T", where T is the name of the member function's class) as if it had been implicitly declared, and~~
>   - not have default arguments.

Add a new paragraph below 11.4.2 [dcl.fct.def.default] paragraph 1:

> The type of an explicitly defaulted function is allowed to differ from the type it would have if it were implicitly declared, as follows:
>
> - it may have differing ref-qualifiers; and
> - in the case of a copy constructor or copy assignment operator, the parameter type may be "reference to non-const T", where T is the name of the member function's class.
>
> If the type differs in any other way, then:
>
> - if the function is explicitly defaulted on its first declaration, it is defined as deleted;
> - otherwise, the program is ill-formed.

Delete 11.4.2 [dcl.fct.def.default] paragraph 3:

> 3. ~~If a function that is explicitly defaulted is declared with a *noexcept-specifier* that does not produce the same exception specification as the implicit declaration ([except.spec]), then~~
>   - ~~if the function is explicitly defaulted on its first declaration, it is defined as deleted;~~
>   - ~~otherwise, the program is ill-formed.~~

Issues Resolved

This proposal would resolve Core Issues #1331 and #1426, both of which are in Extension status, and are tracked by corresponding Evolution Issues #101 and #103, respectively.